A D PATEL INSTITUTE OF TECHNOLOGY
THE CHARUTAR VIDYA MANDAL
DEPT. OF INFORMATION TECHNOLOGY

GRADE CALCULATOR SYSTEM

A Design and Analysis of Algorithms (DAA) Project Report

A C++ application for calculating total marks, percentage, grades and result status

A report submitted

In partial fulfilment of the requirements for the Degree of

Bachelor of Technology

In

Information Technology

Semester – V

Subject:

Design and Analysis of Algorithms (202045601)

By

PATEL DARSH (12502080603004)

Faculty Guide

Minakshi Patil

Assistant Professor

Academic Year 2026

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121

Anand (Gujarat), India

CERTIFICATE

This is to certify that the project work embodied in this report entitled “Grade Calculator System” was prepared by Patel Darsh (Enrollment No.: 12502080603004) at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree in Information Technology.

The project demonstrates the use of basic algorithmic concepts and C++ programming to accept subject marks, validate input, calculate total marks and percentage, determine a grade, and display the result status. The implementation is an educational prototype intended for academic learning.

Date: _____

Place: New Vallabh Vidyanagar

Lab Teacher	**Head of Department**
Minakshi Patil Assistant Professor	Dr. Priyang Bhatt Associate Professor & Head

DECLARATION

I, Patel Darsh (Enrollment No.: 12502080603004), declare that this report entitled “Grade Calculator System” has been prepared by me for the subject Design and Analysis of Algorithms (202045601) in Semester V of the Bachelor of Technology programme in Information Technology.

The project presents an educational C++ implementation for entering marks, checking that they are within the permitted range, calculating total and percentage, assigning a grade according to the grading rules defined in this report, and displaying pass/fail status. The work is submitted for academic learning and does not claim to be an official institute grading application.

Where general programming concepts are based on published material, the relevant sources are acknowledged in the References section.

Date: _____

Place: New Vallabh Vidyanagar

PATEL DARSH
(12502080603004)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my subject teacher and the Department of Information Technology for providing me with the opportunity to develop this Design and Analysis of Algorithms project.

Working on the Grade Calculator System helped me understand how a real-world task can be divided into smaller steps: accepting input, validating values, performing arithmetic operations, applying conditional logic and presenting a clear result. It also strengthened my understanding of functions, loops, arrays, decision structures and testing in C++.

I am thankful to my faculty guide, Minakshi Patil, and everyone who supported my learning and helped me improve the project. I also acknowledge the C++ language references and educational materials listed in the References section.

PATEL DARSH
(12502080603004)

TABLE OF CONTENTS

Section	Page (approx.)
Abstract	6
Chapter 1: Introduction	7
Chapter 2: Problem Statement and Objectives	8
Chapter 3: Theoretical Background	9
Chapter 4: System Requirements and Design	10
Chapter 5: Algorithm and Pseudocode	11
Chapter 6: Grade Rules and Calculations	12
Chapter 7: C++ Implementation	13
Chapter 8: Testing and Results	15
Chapter 9: Sample Execution	16
Chapter 10: Complexity Analysis and Correctness	17
Chapter 11: Limitations, Future Scope and Conclusion	18
References and Appendix	19

Note: Page numbers are approximate and may change slightly when the document is edited or opened in a different version of Word.

ABSTRACT

A Grade Calculator System is a simple application that helps calculate a student's academic result from marks obtained in multiple subjects. Manual calculations can take time and may lead to arithmetic mistakes, especially when totals, percentages and grade categories must be determined repeatedly. A computer-based solution can perform these calculations consistently and display the result in a readable format.

This report describes the analysis, design and implementation of a console-based Grade Calculator System using C++. The program accepts marks for a fixed set of subjects, validates that each mark falls within the permitted range, calculates the total and percentage, assigns a grade using the grading scale defined in the project, and determines the pass/fail status using the stated minimum-mark rule.

The project demonstrates fundamental algorithmic concepts including sequential processing, loops, conditional statements, input validation, arithmetic operations and modular programming. It includes pseudocode, source code, test cases, sample output, complexity analysis, limitations and possible future enhancements. The grading scale is illustrative and should be replaced with the official rules of an institution when used for a real academic record.

Keywords: grade calculator, C++, marks, percentage, conditional statements, algorithm, validation, result processing.

CHAPTER 1: INTRODUCTION

1.1 Background

Students commonly need to calculate total marks, percentages and grade categories across several subjects. These calculations can be completed manually, but repeated arithmetic and inconsistent use of grading rules may produce errors. A Grade Calculator System automates these steps and presents the result in a consistent format.

1.2 Role of Algorithms

An algorithm is a finite sequence of well-defined steps for solving a problem. In this project, the algorithm reads subject marks, checks their validity, calculates the total, derives the percentage, applies grading conditions and prints the result.

- Accuracy: applies the same arithmetic and rules to every valid input.
- Efficiency: calculates results quickly for a fixed set of subjects.
- Consistency: uses one defined grading scale for all calculations.
- Clarity: displays individual marks, total, percentage, grade and result status.

1.3 Why This Topic Matters

A grade calculator is a practical example of how programming can automate a common administrative task. It uses concepts introduced in introductory programming and algorithm analysis, making it suitable for demonstrating input/output, selection, iteration, functions and basic testing.

1.4 Project Overview

The prototype uses five subjects, each marked out of 100. It calculates the total out of 500 and percentage. The example grading scale is: A+ for 90–100%, A for 80–89.99%, B for 70–79.99%, C for 60–69.99%, D for 40–59.99%, and F below 40%. A student passes only when every subject has at least 35 marks. These thresholds are illustrative, not official institute policy.

1.5 Report Organisation

The following chapters define the problem and objectives, explain the relevant concepts, describe system design and grade rules, present the algorithm and C++ code, test representative cases, analyse complexity and conclude with limitations and future scope.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Design a program that accepts marks for five subjects and produces a student's total marks, percentage, grade and pass/fail status. The program must reject marks outside the allowed range and must use clearly defined grading rules.

2.2 Objectives

- Accept marks for five subjects, each out of 100.
- Validate input so that marks are between 0 and 100 inclusive.
- Calculate total marks and percentage correctly.
- Assign a grade according to a stated grading scale.
- Determine pass/fail status using the minimum subject-mark requirement.
- Display the results in a clear, understandable format.
- Test normal, boundary and invalid-input cases.
- Analyse the time and space complexity of the solution.

2.3 Scope

The project is a console-based educational prototype. It processes one student's marks in a single run and uses a fixed number of subjects. It does not connect to a student information system, store records in a database or implement institute-specific rules.

2.4 Success Criteria

- Valid marks are accepted and invalid marks are rejected.
- The total equals the sum of the five subject marks.
- Percentage is calculated as total divided by maximum marks, multiplied by 100.
- Grade boundaries are applied in the correct order.
- A student with any subject below 35 is marked as FAIL.
- Output is readable and repeatable for the same input.

2.5 Assumptions

There are five subjects with equal maximum marks of 100 each. All subjects have equal weight. The example pass threshold is 35 marks per subject. Grade categories use the overall percentage, but pass/fail status is determined separately from subject-level minimum marks.

CHAPTER 3: THEORETICAL BACKGROUND

3.1 What Is a Grade Calculator?

A grade calculator is a program that converts numerical assessment marks into summary information such as total, percentage, grade and result status. The exact rules vary by school, college, course and examination scheme.

3.2 Input and Output

Input consists of marks for five subjects. Output consists of each entered mark, total marks out of 500, percentage, grade and pass/fail status. Clear prompts and labels reduce confusion for the user.

3.3 Arithmetic Operations

The total is the sum of all subject marks. Percentage is calculated using the formula:

Percentage = (Total Marks / Maximum Marks) × 100

For five subjects out of 100, the maximum total is 500. For example, a total of 400 gives a percentage of (400 / 500) × 100 = 80%.

3.4 Conditional Statements

An if-else ladder checks grade boundaries in descending order. Checking the highest threshold first prevents a high percentage from being assigned to a lower category. A separate condition checks whether every subject meets the minimum passing mark.

3.5 Loops and Arrays

An array stores the five subject marks. A loop reads each mark and another loop can sum the values. This reduces repeated code and makes the program easier to maintain if the subject count changes.

3.6 Input Validation

Validation checks whether entered marks are within the allowed range. In the prototype, the user is asked to re-enter a mark if it is below 0 or above 100. Production software should also handle non-numeric input and institution-specific exceptional cases.

3.7 Responsible Use

Academic results are sensitive personal information. A real system should protect student records, limit access, avoid exposing marks to unauthorised people and clearly communicate which grading rules are being used.

CHAPTER 4: SYSTEM REQUIREMENTS AND DESIGN

4.1 Hardware Requirements

- Desktop computer or laptop.
- Keyboard and display.
- Basic memory and storage sufficient to run a C++ compiler.

4.2 Software Requirements

- Operating system: Windows, Linux or macOS.
- C++ compiler supporting standard C++ (for example, g++).
- Text editor or IDE such as Visual Studio Code, Code::Blocks or another C++ IDE.

4.3 Functional Requirements

ID	Requirement	Description
FR1	Read marks	Accept marks for five subjects.
FR2	Validate marks	Only accept values from 0 to 100.
FR3	Calculate total	Add the five subject marks.
FR4	Calculate percentage	Convert total to a percentage of 500.
FR5	Assign grade	Use the defined percentage thresholds.
FR6	Determine result	Pass only if all subject marks are at least 35.
FR7	Display output	Show marks and calculated result details.

4.4 Non-Functional Requirements

- Usability: prompts and output should be simple and clear.
- Reliability: the same valid input should produce the same result.
- Maintainability: grading rules should be easy to update.
- Portability: use standard C++ features available on common platforms.

4.5 System Flow

1. Start the program.
2. Read marks for each of the five subjects.
3. Validate each mark; request a valid value when necessary.
4. Calculate total marks and percentage.
5. Assign a grade from the percentage.
6. Check whether every subject meets the passing threshold.
7. Display marks, total, percentage, grade and result status.
8. End the program.

4.6 Data Design

Variable	Type	Purpose

marks[5]	int array	Stores the five subject marks.
total	int	Stores the sum of all marks.
percentage	double	Stores percentage with decimal precision.
grade	char/string	Stores the assigned grade category.
pass	bool	Tracks whether all subjects meet the pass threshold.

CHAPTER 5: ALGORITHM AND PSEUDOCODE

5.1 Algorithm

9. Begin.
10. Set total = 0 and pass = true.
11. For each subject, read its mark.
12. If a mark is less than 0 or greater than 100, ask for it again.
13. Add the valid mark to total.
14. If the mark is less than 35, set pass = false.
15. Calculate percentage = total × 100 / 500.
16. Use the percentage to assign A+, A, B, C, D or F.
17. If pass is false, display FAIL; otherwise display PASS.
18. Print all marks, total, percentage, grade and result.
19. Stop.

5.2 Pseudocode

```
START
  total ← 0
  pass ← TRUE
  FOR i ← 0 TO 4
      REPEAT
          INPUT marks[i]
      UNTIL marks[i] >= 0 AND marks[i] <= 100
      total ← total + marks[i]
      IF marks[i] < 35 THEN
          pass ← FALSE
      ENDIF
  ENDFOR
  percentage ← (total / 500) × 100
  IF percentage >= 90 THEN grade ← "A+"
  ELSE IF percentage >= 80 THEN grade ← "A"
  ELSE IF percentage >= 70 THEN grade ← "B"
  ELSE IF percentage >= 60 THEN grade ← "C"
  ELSE IF percentage >= 40 THEN grade ← "D"
  ELSE grade ← "F"
  IF pass THEN result ← "PASS"
  ELSE result ← "FAIL"
  DISPLAY marks, total, percentage, grade, result
STOP
```

5.3 Flow of Control

The algorithm has three main phases: input and validation, calculation, and result classification. The input loop guarantees that only in-range marks are used. The calculation phase derives total and percentage. The decision phase assigns the grade and independently checks the pass requirement.

5.4 Design Choice

The program uses a fixed-size array because the prototype contains five subjects. A loop avoids duplicating input and addition statements. A separate grading function could be introduced in a larger version to make testing and maintenance easier.

CHAPTER 6: GRADE RULES AND CALCULATIONS

6.1 Example Grading Scale

Percentage range	Grade	Interpretation
90% to 100%	A+	Excellent
80% to less than 90%	A	Very Good
70% to less than 80%	B	Good
60% to less than 70%	C	Satisfactory
40% to less than 60%	D	Needs Improvement
Below 40%	F	Low overall score

Important: This scale is created only for the project demonstration. Confirm the official grading and passing rules with the relevant department before using a calculator for actual academic results.

6.2 Total Marks

Total Marks = Subject 1 + Subject 2 + Subject 3 + Subject 4 + Subject 5.

6.3 Percentage

Percentage = (Total Marks / 500) × 100. The maximum total is 500 because each of the five subjects is out of 100.

6.4 Pass/Fail Rule

The example rule is that each subject must have at least 35 marks. If even one subject is below 35, the overall result is FAIL, regardless of the overall percentage. Otherwise, the result is PASS. This rule is separate from the grade label.

6.5 Worked Example

Subject	Marks out of 100
Subject 1	85
Subject 2	78
Subject 3	91
Subject 4	72
Subject 5	84
Total	410 / 500
Percentage	82%
Grade	A
Result	PASS

Calculation: 85 + 78 + 91 + 72 + 84 = 410. Percentage = (410 / 500) × 100 = 82%. Since every subject mark is at least 35, the result is PASS and the grade is A.

CHAPTER 7: C++ IMPLEMENTATION

7.1 Program Structure

Component	Purpose
main()	Controls input, calculations and output.
marks array	Stores marks for the five subjects.
input loop	Reads and validates each subject mark.
total calculation	Adds valid marks.
percentage calculation	Converts total to percentage.
grade conditions	Selects the grade category.
pass flag	Tracks whether every subject meets the pass mark.

7.2 C++ Source Code

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    const int SUBJECTS = 5;
    const int MAX_MARKS = 100;
    const int PASS_MARK = 35;
    int marks[SUBJECTS];
    int total = 0;
    bool pass = true;
    double percentage;
    string grade;

    cout << "GRADE CALCULATOR SYSTEM\n";
    cout << "Enter marks for five subjects (0-100).\n";

    for (int i = 0; i < SUBJECTS; i++) {
        do {
            cout << "Subject " << i + 1 << ": ";
            cin >> marks[i];
            if (marks[i] < 0 || marks[i] > MAX_MARKS) {
                cout << "Invalid marks. Enter a value from 0 to 100.\n";
            }
        } while (marks[i] < 0 || marks[i] > MAX_MARKS);

        total += marks[i];
        if (marks[i] < PASS_MARK) {
            pass = false;
        }
    }

    percentage = (static_cast<double>(total) /
                  (SUBJECTS * MAX_MARKS)) * 100.0;

    if (percentage >= 90) grade = "A+";
    else if (percentage >= 80) grade = "A";
    else if (percentage >= 70) grade = "B";
    else if (percentage >= 60) grade = "C";
    else if (percentage >= 40) grade = "D";
    else grade = "F";
```

```
    cout << fixed << setprecision(2);
    cout << "\n----- RESULT -----\n";
    for (int i = 0; i < SUBJECTS; i++) {
        cout << "Subject " << i + 1 << ": " << marks[i] << "/100\n";
    }
    cout << "Total: " << total << "/" << SUBJECTS * MAX_MARKS << "\n";
    cout << "Percentage: " << percentage << "%\n";
    cout << "Grade: " << grade << "\n";
    cout << "Result: " << (pass ? "PASS" : "FAIL") << "\n";
    return 0;
}
```

7.3 Compilation and Execution

```
g++ grade_calculator.cpp -o grade_calculator
./grade_calculator
```

7.4 Explanation of the Code

- Constants define the number of subjects, maximum marks and passing mark.
- A do-while loop repeats input when the entered mark is outside 0–100.
- The total is updated after each valid mark is accepted.
- The pass flag becomes false if any subject mark is below 35.
- The percentage is calculated using floating-point arithmetic.
- An if-else ladder assigns the grade, and the conditional operator prints PASS or FAIL.

7.5 Implementation Notes

This demonstration assumes the user enters numeric values. Non-numeric input would require additional stream-error handling in a more robust version. The program uses a fixed subject count and illustrative grading rules, both of which can be changed to meet a specific syllabus.

CHAPTER 8: TESTING AND RESULTS

8.1 Purpose of Testing

Testing checks that calculations are correct, boundary values are handled properly, invalid marks are rejected, grade categories are assigned correctly and the pass/fail rule is applied independently.

8.2 Test Cases

Test	Input / Situation	Expected behaviour
T1	100, 100, 100, 100, 100	Total 500, 100%, A+, PASS
T2	85, 78, 91, 72, 84	Total 410, 82%, A, PASS
T3	70, 65, 60, 55, 50	Total 300, 60%, C, PASS
T4	40, 40, 40, 40, 40	Total 200, 40%, D, PASS
T5	20, 80, 80, 80, 80	Grade based on 68%; result FAIL due to one subject below 35
T6	0, 0, 0, 0, 0	Total 0, 0%, F, FAIL
T7	Input mark -1 or 101	Program requests a valid mark again
T8	Marks exactly 35 in each subject	Pass condition is satisfied

8.3 Boundary Testing

Boundary testing is important around grade cutoffs and the valid mark range. Values 0 and 100 should be accepted; -1 and 101 should be rejected. Percentages at 40, 60, 70, 80 and 90 should fall into the category whose lower bound is that value.

8.4 Observations

- The total is the sum of all valid subject marks.
- The percentage cannot exceed 100 when all marks are in range.
- The grade depends on the overall percentage.
- Pass/fail depends on each subject's minimum mark, not only the total.
- Invalid out-of-range marks are not added to the total.

8.5 Test Limitations

The test cases describe expected behaviour. To provide execution evidence for submission, compile and run the program on your own computer and insert a genuine screenshot of the output if your instructor requires it.

CHAPTER 9: SAMPLE EXECUTION

9.1 Sample Input

```
Subject 1: 85
Subject 2: 78
Subject 3: 91
Subject 4: 72
Subject 5: 84
```

9.2 Sample Output

```
GRADE CALCULATOR SYSTEM
Enter marks for five subjects (0-100).

----- RESULT -----
Subject 1: 85/100
Subject 2: 78/100
Subject 3: 91/100
Subject 4: 72/100
Subject 5: 84/100
Total: 410/500
Percentage: 82.00%
Grade: A
Result: PASS
```

9.3 Interpreting the Output

The five marks add up to 410. Dividing 410 by 500 and multiplying by 100 gives 82%. Under the illustrative scale, 82% maps to grade A. Every subject has at least 35 marks, so the result is PASS.

9.4 Invalid Input Example

If a user enters 105 for a subject, the program displays an invalid-marks message and asks for another value. The invalid value is not accepted as the subject mark.

9.5 Evidence to Add

Insert a screenshot of the actual program running on your computer here if execution evidence is required. Do not label a typed sample output as a screenshot of a real execution.

CHAPTER 10: COMPLEXITY ANALYSIS AND CORRECTNESS

10.1 Time Complexity

Let n be the number of subjects. The input and validation loop processes each subject a constant number of times when valid input is supplied. Total calculation and pass checking each take constant time per subject. Therefore, the overall time complexity is O(n). For this prototype, n = 5, so the work is effectively constant.

10.2 Space Complexity

The program stores n marks in an array and uses a fixed number of additional variables. The space complexity is O(n), dominated by the marks array.

10.3 Correctness of Total

Initially, total is zero. Each valid mark is added exactly once during the loop. After all n subjects have been processed, total equals the sum of all accepted subject marks.

10.4 Correctness of Percentage

The percentage is calculated as total divided by the maximum possible total, multiplied by 100. Because every accepted mark lies between 0 and 100, the total lies between 0 and 500, and the percentage lies between 0 and 100.

10.5 Correctness of Grade Assignment

The conditions are checked from the highest percentage threshold to the lowest. The first satisfied condition determines the grade. Therefore, a percentage of 90 or above receives A+, while a value below 40 receives F.

10.6 Correctness of Pass/Fail

The pass flag begins as true. Whenever a subject mark is below 35, the flag is set to false and is never reset to true. Consequently, the final result is PASS only when every subject mark is at least 35.

10.7 Performance Notes

The system is designed for a small fixed number of subjects, so its processing cost is minimal. For larger systems, performance would still be straightforward, while data security, storage, input reliability and institution-specific rule management would become more important.

CHAPTER 11: LIMITATIONS, FUTURE SCOPE AND CONCLUSION

11.1 Limitations

- The program supports only five subjects.
- The grading scale and pass threshold are illustrative.
- The program processes one student at a time.
- Marks are not saved to a file or database.
- Non-numeric input is not fully handled.
- There is no login, report generation or graphical interface.

11.2 Future Enhancements

- Allow the user to specify the number and names of subjects.
- Load official grading rules from a configuration file.
- Support different maximum marks and subject weightings.
- Handle non-numeric input and unexpected end-of-file safely.
- Store student records in a database with suitable access controls.
- Generate printable mark sheets and export results to CSV or PDF.
- Create a graphical user interface or web-based version.
- Add unit tests and automated validation for every grade boundary.

11.3 Conclusion

The Grade Calculator System demonstrates how a common academic calculation can be represented as a simple algorithm and implemented in C++. It accepts marks, validates the allowed range, calculates total and percentage, assigns a grade and determines pass/fail status. Loops reduce repeated code, conditional statements implement grading rules, and a fixed-size array stores subject marks.

The project reinforces important programming and algorithm-analysis concepts, including input validation, arithmetic operations, selection, iteration, correctness and complexity. Although the current implementation is a classroom prototype, it provides a foundation for a more complete result-processing application. Before any real use, the grading scale, passing criteria, privacy safeguards and data-handling requirements must be confirmed with the institution.

REFERENCES AND APPENDIX

References

20. ISO/IEC 14882. Programming Languages — C++ (C++ language standard).
21. cppreference.com. C++ language and standard library reference. https://en.cppreference.com/
22. Stroustrup, B. The C++ Programming Language. Addison-Wesley.
23. Deitel, P. and Deitel, H. C++ How to Program. Pearson.
24. Course notes and classroom material for Design and Analysis of Algorithms, Department of Information Technology, A D Patel Institute of Technology.

Appendix A — Core Calculation

```
total = marks[0] + marks[1] + marks[2] + marks[3] + marks[4];
percentage = (total / 500.0) * 100.0;
```

Appendix B — Submission Checklist

- Confirm the student name and enrollment number on the cover, certificate and declaration.
- Confirm the subject code, faculty guide and department details with the department.
- Compile and run the C++ program.
- Test the grade boundaries and invalid input cases.
- Insert a genuine execution screenshot if required.
- Update the table of contents and page numbers after final formatting.
- Replace the example grading scale with official rules if instructed.